ACE ENGINEERING COLLEGE (AN AUTONOMOUS INSTITUTION)
Department of Computer Science & Engineering (IoT & AI&DS)

MAJOR PROJECT REVIEW — STAGE 1

WORKUP: Virtual Professional Experience Platform

A Simulated Workplace Platform for Practical Software Engineering and Skill Verification

BATCH NO: 05 | TEAM DETAILS:
- Mashetty Rohith (23AG1A7236)
- Chintam Archana (23AG1A7214)
- Namasi Aravind (24AG5A7205)

SUPERVISION & REVIEW:
Internal Guide: Mrs. P. Swaroopa
Assistant Professor, Dept. of CSE (IoT & AI&DS)
Head of Department: Dr. M. V. Vijaya Saradhi
Professor & HoD-CSE (IoT & AI&DS)

Abstract

A virtual workplace platform bridging the gap between classroom coding and industry practice

ACE Engineering College
CSE (IoT & AI&DS)

THE ACADEMIC GAP

Most computer science students learn programming concepts through lectures and isolated coding exercises, but face significant difficulties when transitioning to professional software engineering teams. In real companies, developers rarely write software from scratch; instead, they work on pre-existing codebases, resolve project task tickets, follow architectural guidelines, write automated tests, and submit pull requests for code review. Existing learning platforms focus heavily on syntax and algorithmic puzzles, leaving students with limited exposure to realistic team workflows.

THE WORKUP SOLUTION

This project presents WORKUP, a web-based virtual professional experience platform designed to bridge this practical gap. Instead of solving toy problems, learners enter simulated tech companies (such as FinFlow Technologies) and take on realistic engineering roles. The platform organizes work into project backlogs using a task dependency engine, linking each task to real business scenarios and Architecture Decision Records (ADRs). Learners work inside isolated browser-based development sandboxes where they can edit code, run automated tests locally, and submit their changes for review.

EVALUATION & SKILL PROOF

An automated AI-assisted code review engine evaluates code changes against defined rubrics and provides line-level feedback to guide the learner. As tasks are approved, the platform creates an Evidence-of-Skill portfolio backed by verified Git commits. The system is designed and prototyped using Next.js 14, React.js, TypeScript, Prisma ORM, and Vitest, offering a safe, hands-on environment where students gain realistic workplace experience before starting their careers.

Describe the Identified Problem

Four fundamental challenges students face when transitioning from classroom coding to industry

ACE Engineering College
CSE (IoT & AI&DS)

CHALLENGE 1

Clean-Slate Learning (Toy Problems)

Students usually practice writing small programs from an empty file. In real software jobs, developers almost never write code from scratch. Instead, they must read, understand, debug, and improve large existing project codebases that already have technical debt.

CHALLENGE 2

No Real Architectural Context

Classroom assignments only give simple algorithmic prompts without business background, task tickets, or Architecture Decision Records (ADRs). As a result, students do not learn how to follow company design standards or handle production constraints.

CHALLENGE 3

Lack of In-Depth Code Review

Standard practice websites only test code with simple pass or fail checks. Students rarely experience line-by-line code reviews from senior engineers to learn clean coding habits, edge-case handling, and proper pull request communication.

CHALLENGE 4

Unverifiable Resume Claims

Resumes only show self-written lists of technologies and course certificates. Recruiters and faculty have no practical way to verify a student's actual problem-solving and coding ability on realistic, multi-file engineering tasks.

Existing Systems

Overview of current learning platforms, coding websites, and job simulations

Algorithmic Practice Websites (LeetCode, HackerRank)

Focus on short algorithm puzzles and interview math problems in a single file. They do not expose learners to multi-file codebases, configuration files, or team collaboration.

Video-Based Online Courses (Coursera, Udemy, edX)

Rely primarily on passive video lectures and follow-along tutorials. Students type code alongside instructors without experiencing realistic debugging or architectural decision-making.

Static Job Simulations (Forage)

Provide corporate business scenarios and job briefs, but learner submissions are static documents (PDFs, PPT slides, or spreadsheets) with no live coding, automated testing, or Git version control.

Unguided Cloud IDEs (GitHub Codespaces, Gitpod)

Provide full Linux virtual machines in the browser, but lack curriculum structure, simulated company context, guided task backlogs, or automated rubric-based code evaluations.

Disadvantages of Existing Systems

Key drawbacks that leave fresh computer science graduates unprepared for engineering jobs

ACE Engineering College
CSE (IoT & AI&DS)

- **Clean-Slate Bias:** Students only learn to write code from scratch, leaving them overwhelmed when faced with large production codebases with thousands of existing lines.
- **No Real Business Context:** Assignments lack authentic company background, Architecture Decision Records (ADRs), product requirements, and realistic issue tracking.
- **Binary Pass / Fail Grading:** Current auto-graders only test simple string outputs rather than inspecting code design, maintainability, architectural compliance, and clean code principles.
- **Unverifiable Resumes:** Self-reported technical skills on resumes are easy to exaggerate and offer hiring teams zero evidence of genuine problem-solving ability.
- **Fragmented Learning Tools:** Learners must constantly switch between tutorial videos, coding websites, personal editors, and GitHub without a unified workplace workflow.

Proposed System & Workflow

The WORKUP platform workflow connecting simulated companies, tasks, sandboxes, and reviews

ACE Engineering College
CSE (IoT & AI&DS)

CORE PLATFORM CONCEPT

WORKUP acts as a virtual workplace platform that immerses learners in simulated engineering companies. It provides realistic task tickets, executes code in isolated cloud sandboxes, and delivers automated line-level reviews to help students build practical engineering competence.

END-TO-END 7-STAGE PROCESS FLOW

1. **Career & Role Selection:** Learner chooses a career family and specialized role (e.g., Backend Engineer, Platform SRE).
2. **Simulated Company:** Enters a simulated enterprise (e.g., FinFlow Technologies) with authentic business context.
3. **Task Ticket & ADR:** Receives a Jira-style task ticket with acceptance criteria and Architecture Decision Records.
4. **Isolated Cloud Sandbox:** Opens a zero-setup browser workspace with path-jailed file execution.
5. **Implementation & Tests:** Refactors code and executes in-sandbox automated tests (Vitest) to verify correctness.
6. **AI Code Review:** Submits a pull request; the review engine evaluates code diffs and provides line-level feedback.
7. **Evidence of Skill:** Approved tasks generate an immutable portfolio backed by verified Git commit SHAs.

Advantages of Proposed System

Key practical advantages for students, faculty, and recruiting institutions

Realistic Workplace Experience

Learners work inside simulated companies (such as FinFlow Technologies) and solve genuine business problems instead of isolated syntax puzzles.

Connected Project Workflows

Follows the complete engineering cycle from task tickets to testing, pull request review, and portfolio evidence in one platform.

Zero-Setup Cloud Sandboxes

Students write and test code directly in the browser without installing complex tools or dealing with operating system incompatibilities.

Automated In-Sandbox Testing

Built-in test execution (Vitest) allows learners to run tests locally, see immediate feedback, and fix bugs before submitting work.

Line-by-Line Code Review

An automated AI Tech Lead reviewer inspects code diffs and provides specific comments on clean architecture, security, and edge cases.

Verifiable Proof of Skills

Replaces self-reported resume claims with an Evidence-of-Skill portfolio backed by verified Git commit hashes and test logs.

End Users / Target Beneficiaries

Key user roles and institutional groups that benefit from the WORKUP platform

ROLE 1: STUDENTS / LEARNERS

Engineering Students & Career Starters

Explore professional career paths (Software Engineering, DevOps, AI/ML), enter simulated tech companies, work on realistic task tickets, write code in cloud sandboxes, run automated tests locally, receive line-level feedback, and build a verified portfolio of real code commits.

ROLE 2: EDUCATORS / AUTHORS

Faculty & Curriculum Designers

Create simulated companies, design multi-step project task backlogs (DAGs), set up legacy codebases, define automated test suites, and configure evaluation rubrics to assess student learning with zero manual grading overhead.

ROLE 3: RECRUITERS / MENTORS

Hiring Managers & Senior Engineers

Review candidate portfolios backed by verified Git commits and test reports, evaluate practical problem-solving ability, and provide mentor feedback without relying on self-reported resume bullet points.

Technology Stack

Core technologies used in the development and prototyping of WORKUP

ACE Engineering College
CSE (IoT & AI&DS)

TECHNOLOGY	ROLE & PURPOSE IN WORKUP PLATFORM
Next.js 14	Builds the full-stack web application with App Router, server-rendered pages, and secure API routes.
React.js 18	Creates the interactive user interface, including career catalogs, task boards, and workbench panels.
TypeScript	Provides static type-checking across frontend components, API handlers, and backend business logic.
Tailwind CSS	Provides responsive styling and modern layout components across all platform pages.
Prisma ORM	Manages database queries and models 21 relational domain entities (users, companies, tasks, workspaces).
PostgreSQL / SQLite	Stores application data including user accounts, simulated company details, task tickets, and evaluations.
Node.js Sandbox	Runs student code safely in path-jailed workspaces with strict execution timeouts (4000ms).
Vitest	Executes automated unit and integration tests inside the sandbox and checks student code against assertions.
JWT & Bcryptjs	Handles user authentication with encrypted password hashing and secure HTTP-only session tokens.
Git Version Control	Captures code changes (diffs), generates commit hashes, and verifies student portfolios.

Literature Survey (1)

Careful review of published research in software engineering simulation, sandboxes, and education

PAPER TITLE: Comprehensive Evaluation of an Educational Software Engineering Simulation Environment

AUTHOR: Emily Navarro, André van der Hoek | **YEAR:** 2007 (IEEE Transactions on Software Engineering)

STUDY DESCRIPTION & KEY FINDINGS:

This paper presents the design and longitudinal evaluation of SimSE, an educational software engineering simulation environment. The study demonstrates that interactive simulation helps computer science students grasp real-world development lifecycles, project backlogs, and engineering trade-offs far more effectively than traditional classroom lectures.

PURPOSE OF REFERRING THIS PAPER FOR MY PROJECT:

I am referring to this paper to design the simulated company workflow and sprint backlog system for WORKUP. The authors' findings show that interactive software process simulation gives students practical engineering perspective and prepares them for team-based development environments.

Literature Survey (2)

Careful review of published research in software engineering simulation, sandboxes, and education

PAPER TITLE: Gaps Between Industry Expectations and University Preparation in Software Engineering

AUTHOR: Alex Radermacher, Gursimran Walia | **YEAR:** 2013 (ACM SIGCSE Technical Symposium)

STUDY DESCRIPTION & KEY FINDINGS:

This paper investigates industry deficiencies among fresh computer science graduates across more than 50 technology companies. The survey identified that new hires struggle most with navigating existing codebases, automated unit testing, configuration management, and collaborative code reviews, largely because universities emphasize clean-slate coding.

PURPOSE OF REFERRING THIS PAPER FOR MY PROJECT:

I am referring to this paper to establish the core problem statement and functional requirements for WORKUP. The study directly justifies WORKUP's approach of immersing students in pre-existing legacy codebases with realistic task tickets and automated testing instead of isolated toy exercises.

Literature Survey (3)

Careful review of published research in software engineering simulation, sandboxes, and education

ACE Engineering College
CSE (IoT & AI&DS)

PAPER TITLE: **Lightweight Container Isolation for Computing Education**

AUTHOR: Nane Kratzke | **YEAR:** 2018 (IEEE Transactions on Education)

STUDY DESCRIPTION & KEY FINDINGS:

This paper evaluates lightweight container isolation mechanisms for delivering browser-accessible development environments in computer science education. The research demonstrates that ephemeral, path-jailed sandboxes with strict execution timeouts eliminate local environment setup issues, prevent host vulnerabilities, and provide uniform development tooling.

PURPOSE OF REFERRING THIS PAPER FOR MY PROJECT:

I am referring to this paper to design the isolated browser-based development workbench in WORKUP. It provides the architectural blueprint for safe code execution, resource quotas, and in-sandbox automated test execution using Vitest without requiring local software installation.

Literature Survey (4)

Careful review of published research in software engineering simulation, sandboxes, and education

ACE Engineering College
CSE (IoT & AI&DS)

PAPER TITLE: **Automated Code Review in Software Engineering: A Systematic Literature Review**

AUTHOR: Carolin Bogner, Roberto Verdecchia, Patricia Lago | **YEAR:** 2021 (IEEE Access / ACM Computing Surveys)

STUDY DESCRIPTION & KEY FINDINGS:

This systematic review analyzes modern automated code review systems, static analysis pipelines, and rubric-based feedback tools in software development. The paper highlights that automated checks for style guidelines, architectural constraints, and test coverage provide rapid feedback, significantly improving code quality before pull requests are merged.

PURPOSE OF REFERRING THIS PAPER FOR MY PROJECT:

I am referring to this paper to structure the AI Tech Lead review engine in WORKUP. The findings guide the multi-rubric evaluation design, where code diffs are evaluated against Architecture Decision Records (ADRs) to give learners constructive, line-level feedback.

Literature Survey (5)

Careful review of published research in software engineering simulation, sandboxes, and education

PAPER TITLE: **Scaffolding Software Engineering Learning Through Directed Acyclic Graph Workflows**

AUTHOR: David Shepherd, Lori Pollock, Vijay K. Vaishnavi | **YEAR:** 2019 (IEEE Transactions on Learning Technologies)

STUDY DESCRIPTION & KEY FINDINGS:

This study examines instructional scaffolding and task sequencing in software engineering education using Directed Acyclic Graphs (DAGs). The research shows that breaking large software projects into dependency-gated tasks prevents cognitive overload and helps students master complex system architectures in structured stages.

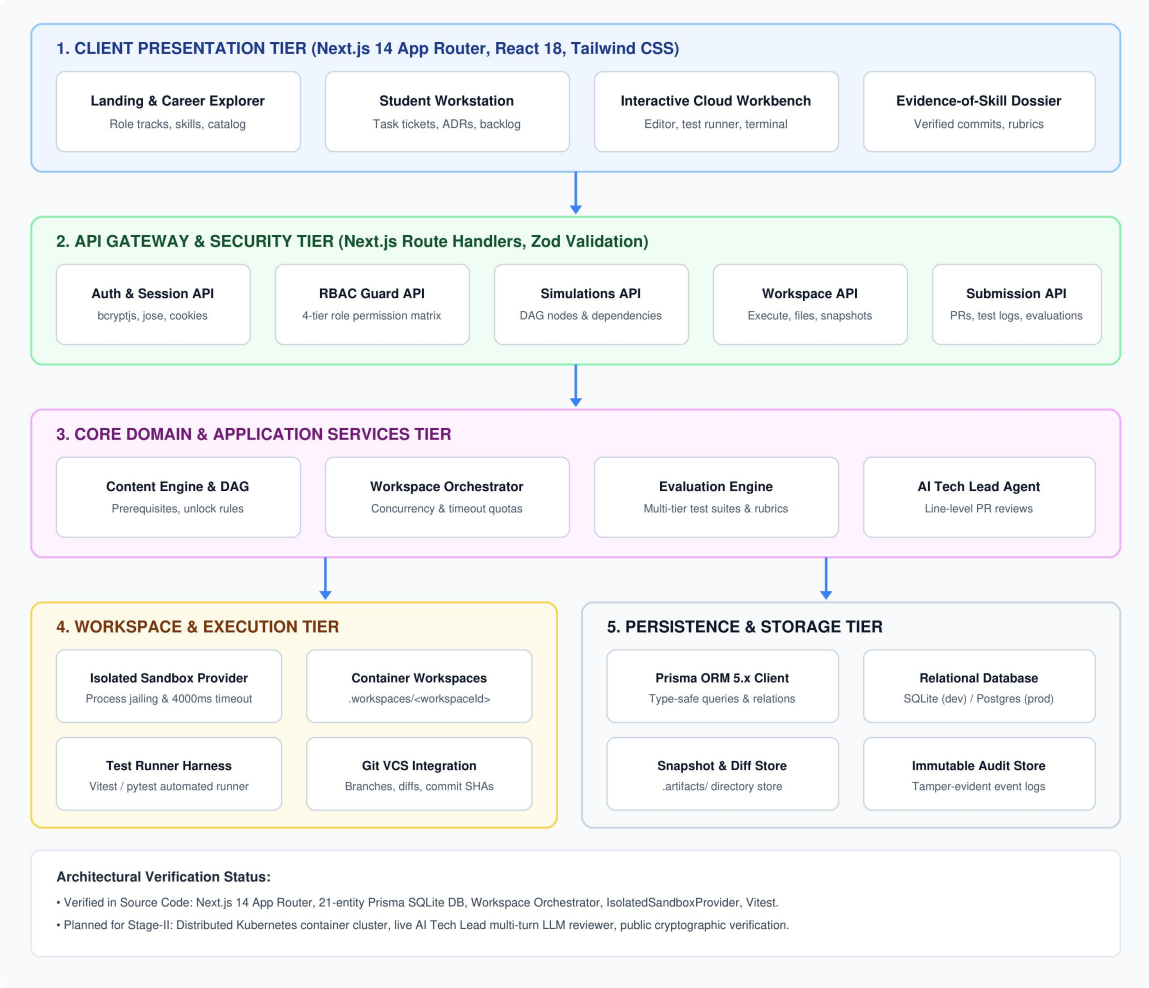
PURPOSE OF REFERRING THIS PAPER FOR MY PROJECT:

I am referring to this paper to design the task dependency engine (DAG) in WORKUP. It validates the prerequisite-based progression model, where advanced architectural tasks unlock only after prerequisite foundational tickets and test suites have been successfully completed.

Architecture Diagram

5-tier architecture connecting presentation, API gateway, core services, sandbox, and storage

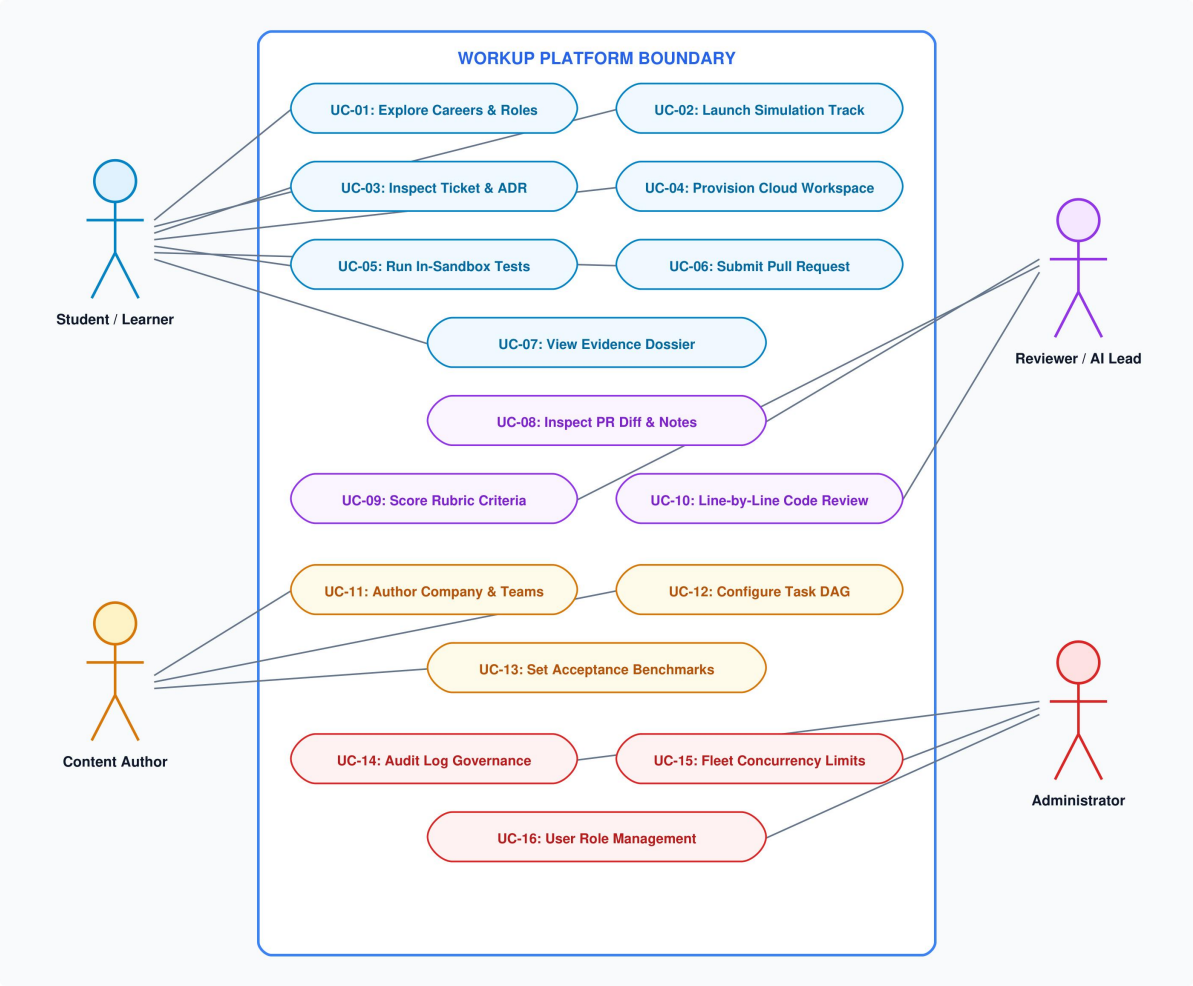
ACE Engineering College
CSE (IoT & AI&DS)



Use Case Diagram

Core platform actors (Student, Author, Reviewer, Admin) interacting with 16 system use cases

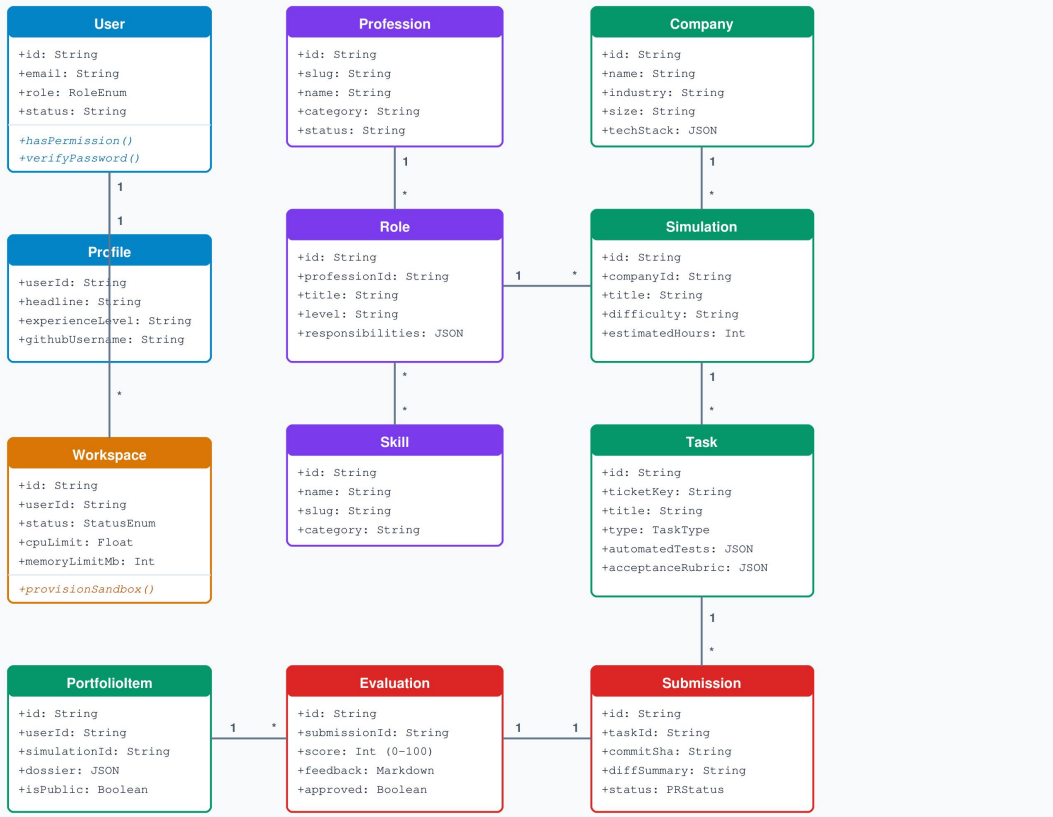
ACE Engineering College
CSE (IoT & AI&DS)



Class Diagram

21-entity domain model capturing users, simulations, tasks, workspaces, submissions, and portfolios

ACE Engineering College
CSE (IoT & AI&DS)



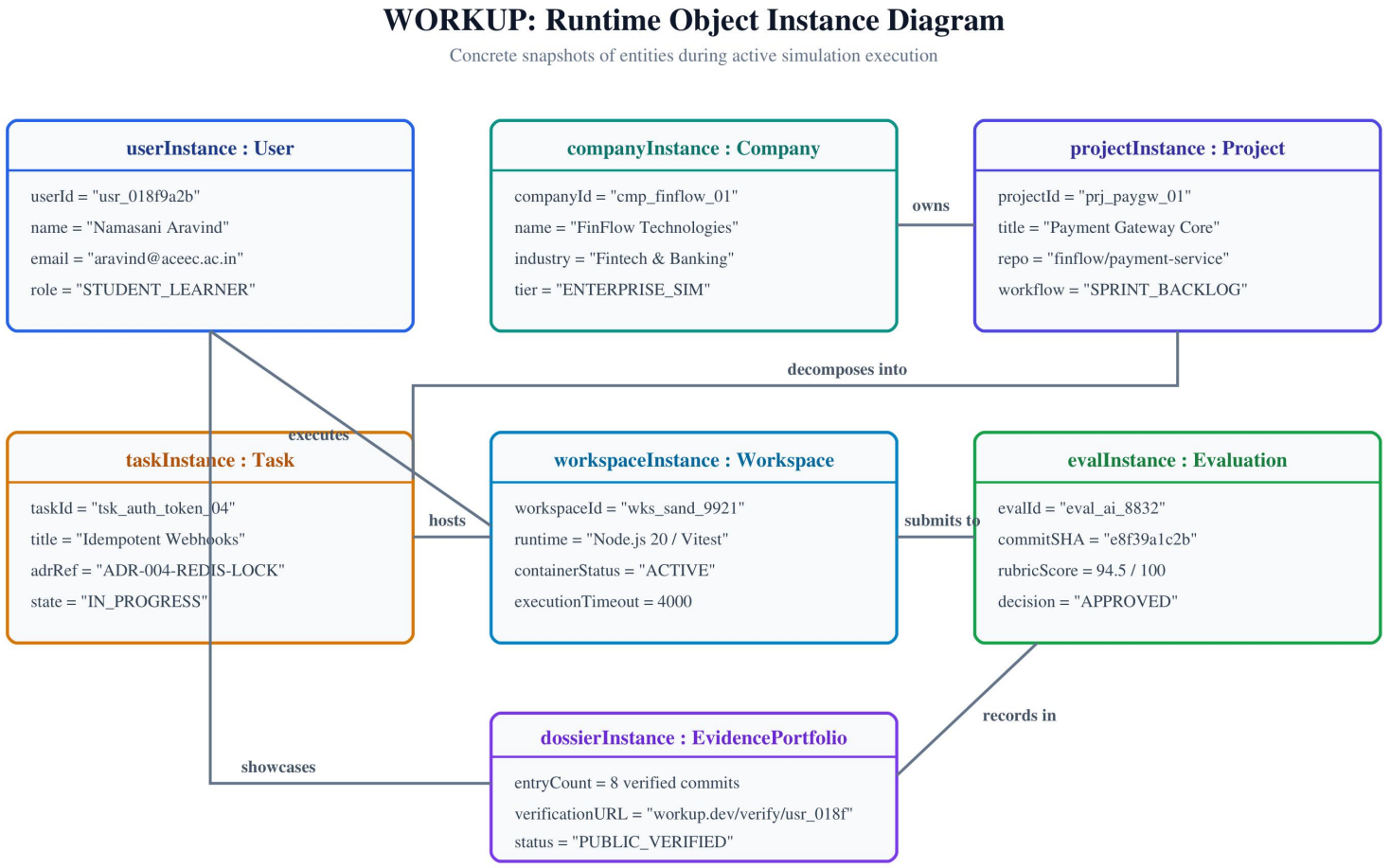
Model Schema Consistency:

Strictly reflects the 21 models defined in prisma/schema.prisma including User, Profession, Role, Company, Simulation, Task, Workspace, Submission, Evaluation, and PortfolioItem.

Object Diagram

Concrete runtime instances of user, company, project, task, workspace, and evaluation records

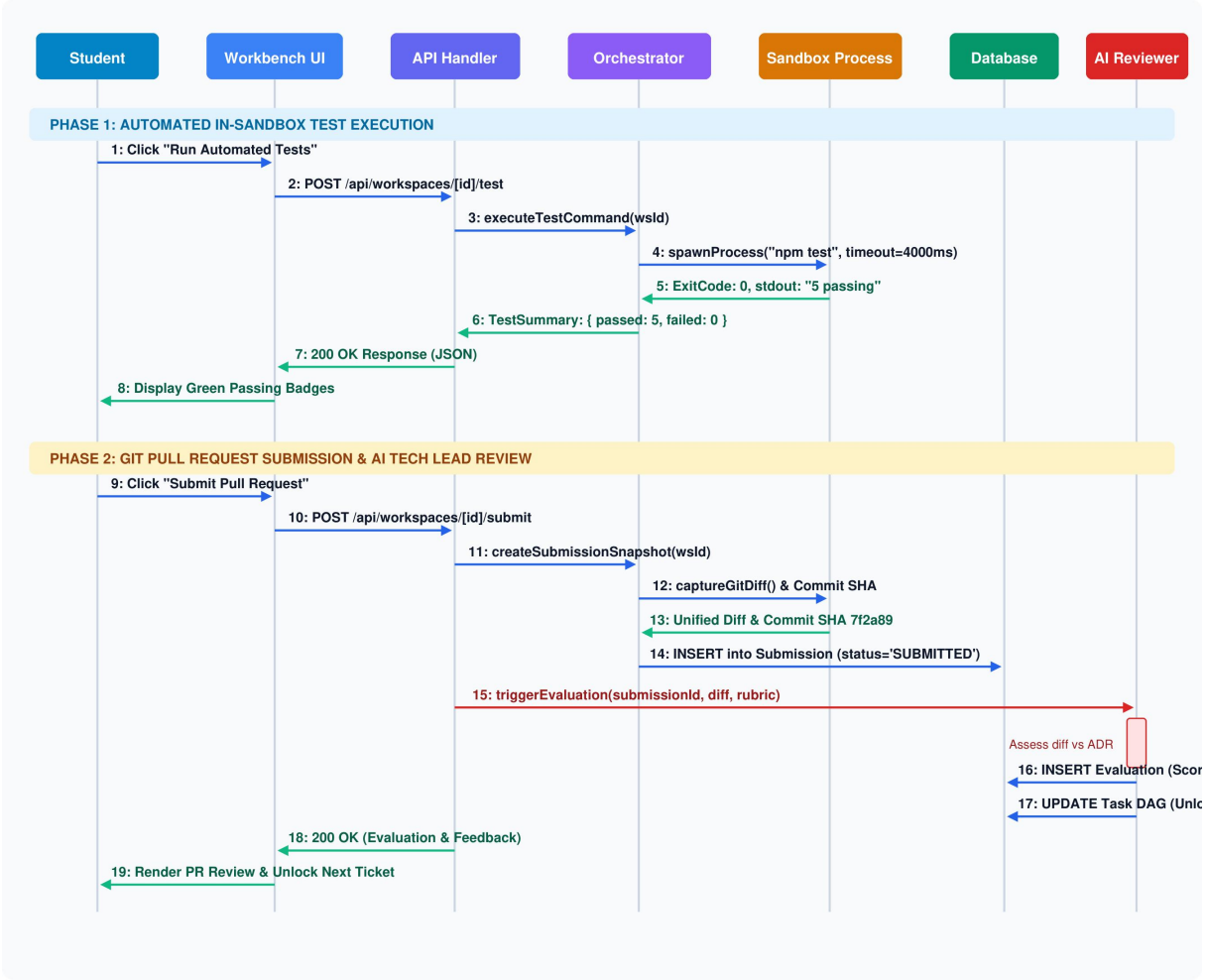
ACE Engineering College
CSE (IoT & AI&DS)



Sequence Diagram

Two-phase sequence flow: Automated In-Sandbox Test Execution and AI PR Review Submission

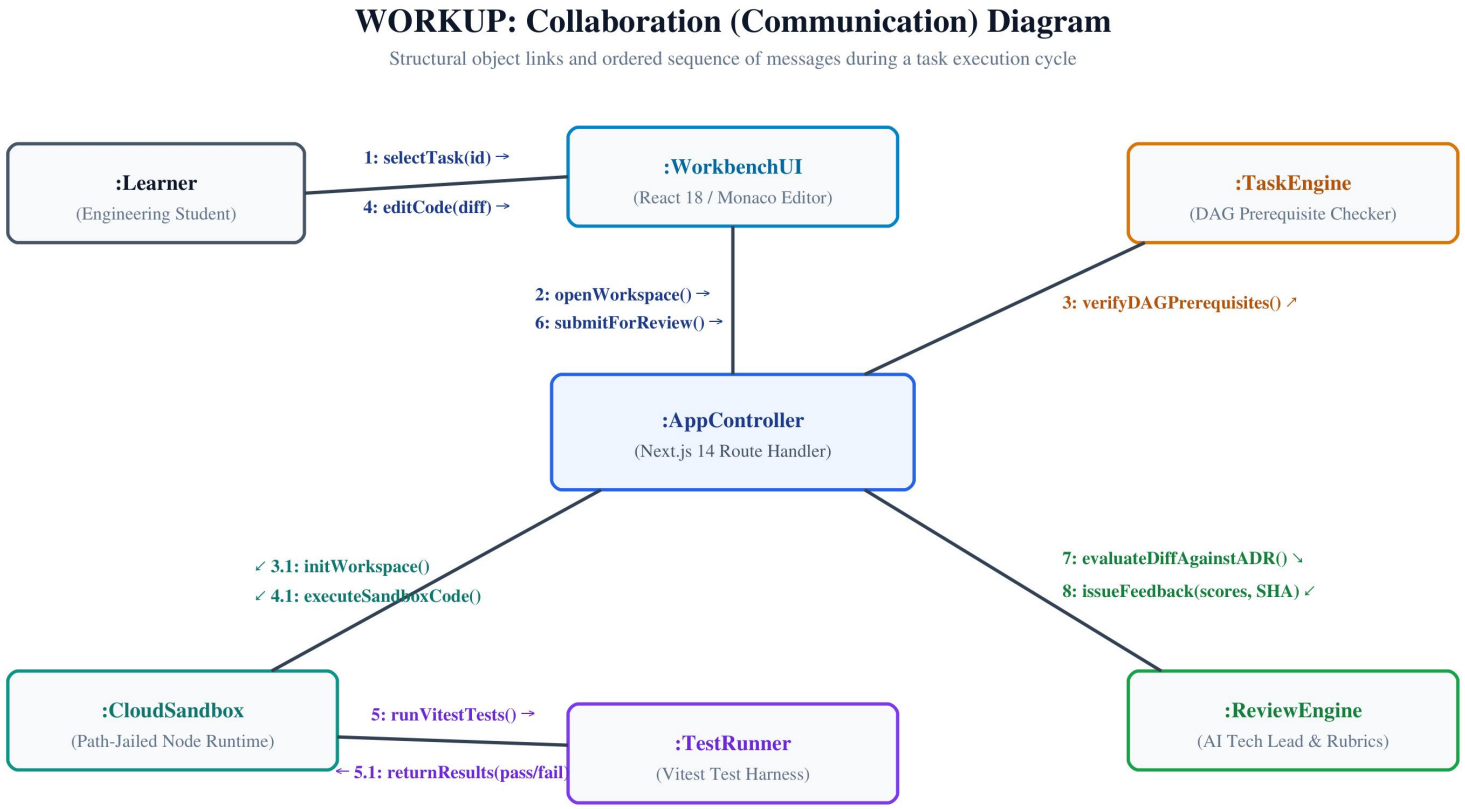
ACE Engineering College
CSE (IoT & AI&DS)



Collaboration Diagram

Object relationships and ordered message flow during an end-to-end task completion cycle

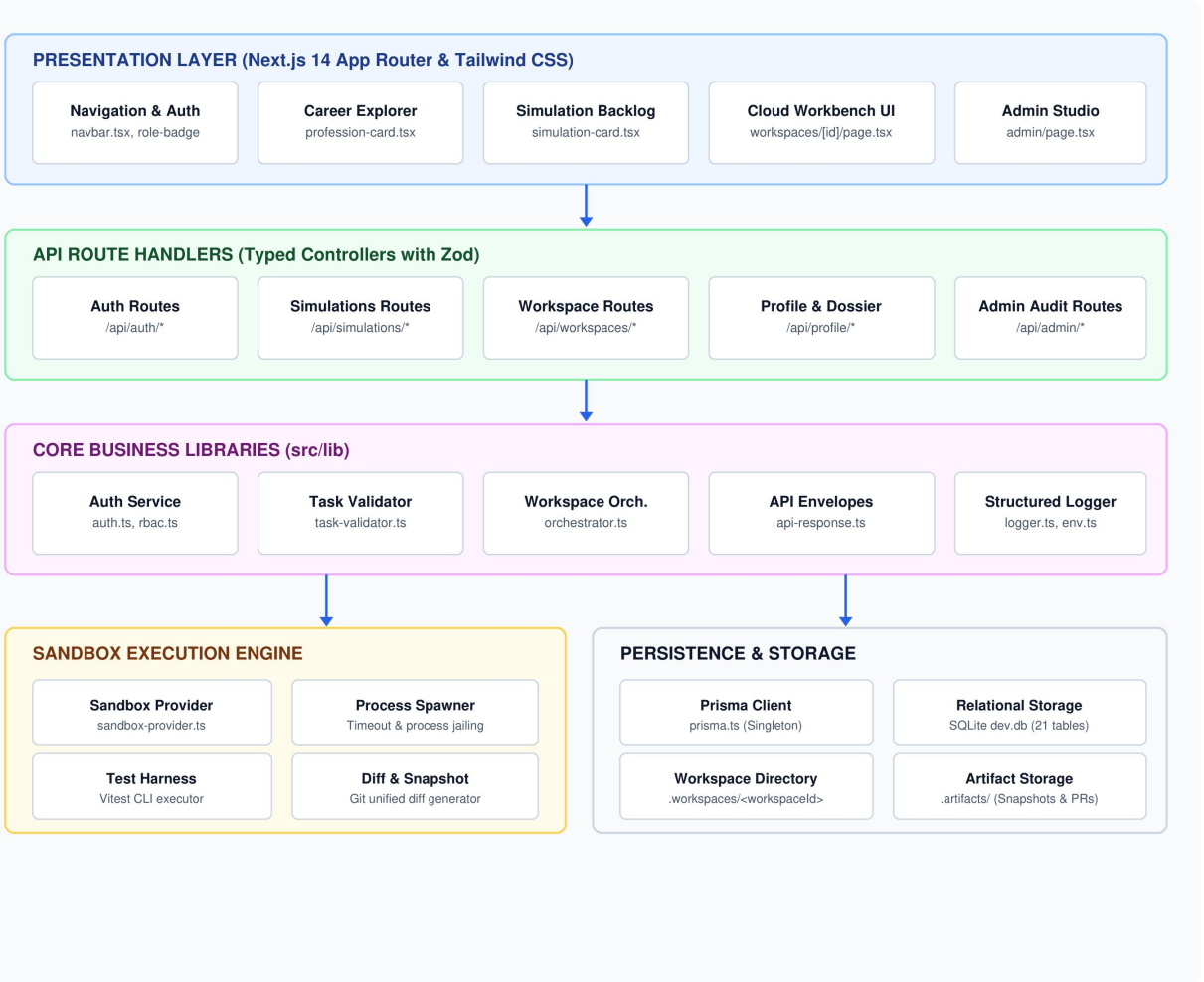
ACE Engineering College
CSE (IoT & AI&DS)



Component Diagram

Layered software components from Next.js UI through API routes, business logic, and sandbox

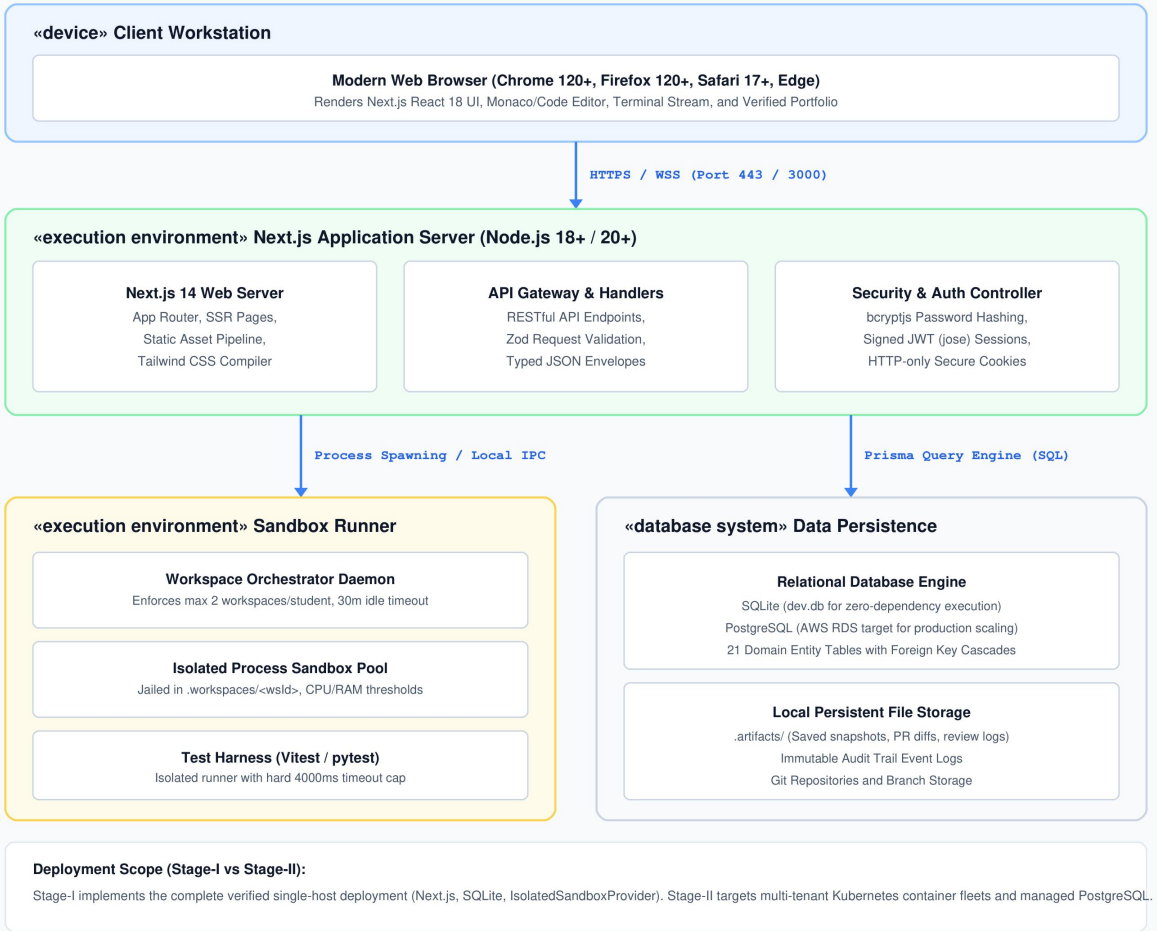
ACE Engineering College
CSE (IoT & AI&DS)



Deployment Diagram

Hardware nodes and execution environments: Browser Client, Web App Server, Sandbox Container Host

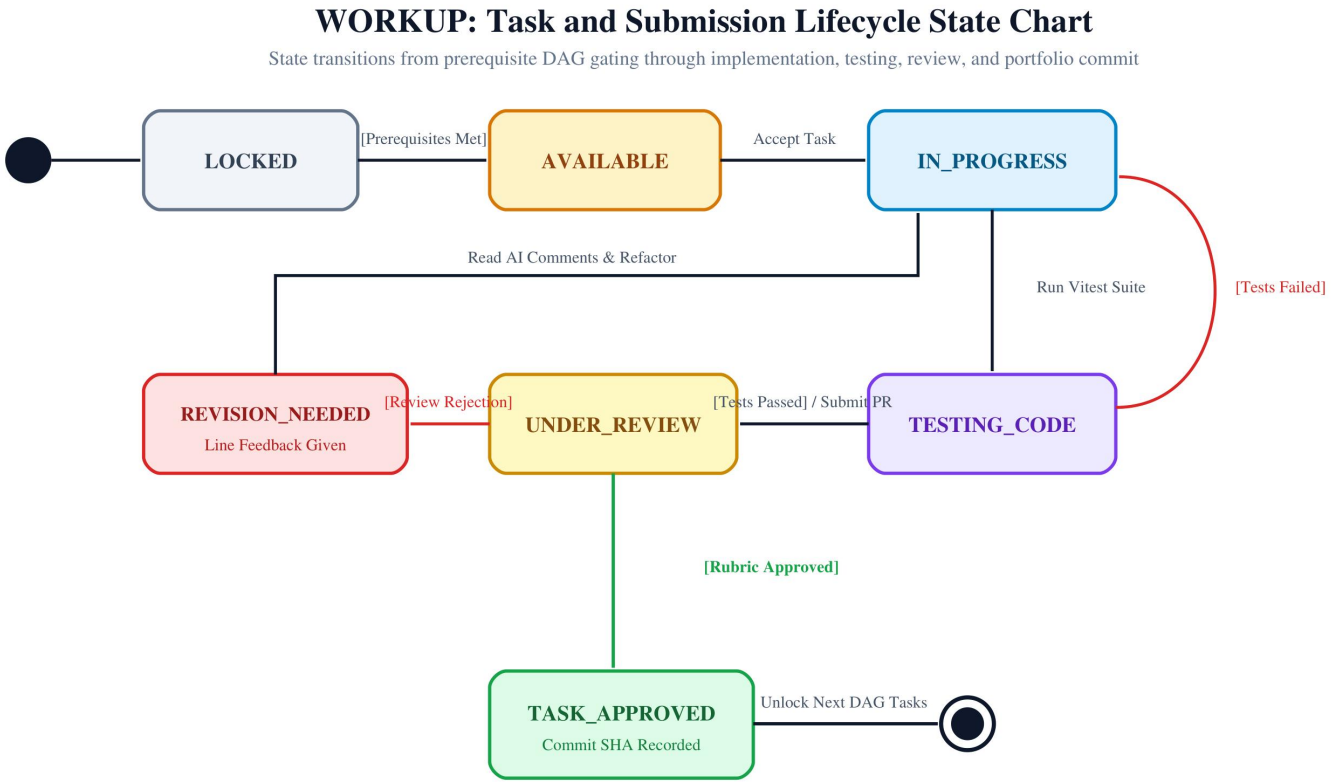
ACE Engineering College
CSE (IoT & AI&DS)



State Chart Diagram

Task & submission state transitions from prerequisite LOCKED to final TASK_APPROVED

ACE Engineering College
CSE (IoT & AI&DS)

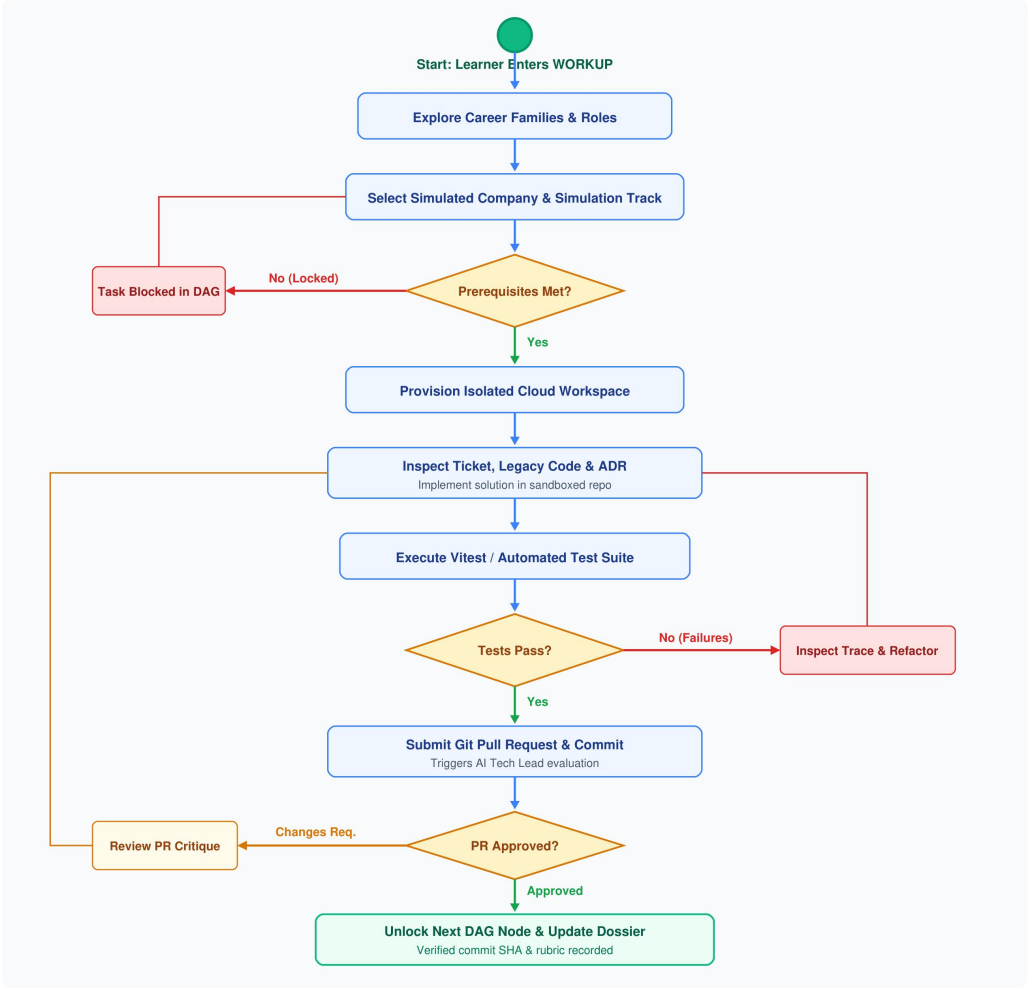


PROCESS WORKFLOW

Activity Diagram

Learner journey from role selection through ticket inspection, sandbox coding, tests, and PR review

ACE Engineering College
CSE (IoT & AI&DS)



Conclusion

Summary of the WORKUP platform's impact, current prototype, and future roadmap

ACE Engineering College
CSE (IoT & AI&DS)

SUMMARY OF PROPOSED SOLUTION

The proposed WORKUP platform introduces a practical and modern approach to bridging the gap between classroom programming and real-world software engineering. By immersing learners in simulated technology companies with authentic task backlogs, Architecture Decision Records (ADRs), and legacy codebases, the system directly addresses the critical limitations of conventional coding websites that rely on isolated toy problems.

CURRENT PROTOTYPE ACHIEVEMENTS

The platform successfully combines zero-setup browser sandboxes, automated in-sandbox testing via Vitest, and AI-assisted code reviews to provide immediate, actionable feedback to learners. As tasks are completed, verifiable Git commit records build an authentic Evidence-of-Skill portfolio for recruiters and educators, ensuring that student abilities are backed by genuine code evidence.

FUTURE ENHANCEMENTS & SCALABILITY

This prototype lays a solid architectural foundation for virtual workplace simulations. Future stages will extend the platform with multi-developer team collaboration, distributed Kubernetes sandbox container clusters, live peer code reviews, and specialized tracks in cloud infrastructure, AI engineering, and cybersecurity.

References

Key peer-reviewed literature and conference proceedings cited in this review

[1] Navarro, E., & van der Hoek, A. (2007). Comprehensive evaluation of an educational software engineering simulation environment. IEEE Transactions on Software Engineering, 33(6), 409-425.

[2] Radermacher, A., & Walia, G. (2013). Gaps between industry expectations and university preparation in software engineering. In Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE), 525-530.

[3] Kratzke, N. (2018). Lightweight container isolation for computing education. IEEE Transactions on Education, 61(2), 127-134.

[4] Bogner, C., Verdecchia, R., & Lago, P. (2021). Automated code review in software engineering: A systematic literature review. IEEE Access, 9, 78241-78262.

[5] Shepherd, D., Pollock, L., & Vaishnavi, V. K. (2019). Scaffolding software engineering learning through directed acyclic graph workflows. IEEE Transactions on Learning Technologies, 12(3), 341-354.

COMPARATIVE MATRIX

Literature Survey Summary Table

Structured comparison of studied papers, key methodologies, findings, and relevance to WORKUP

ACE Engineering College
CSE (IoT & AI&DS)

PAPER / AUTHOR / YEAR	APPROACH	MAIN FINDINGS	RELEVANCE TO WORKUP
Navarro & van der Hoek (2007, IEEE TSE)	SimSE Process Simulation	Active simulation significantly improves software lifecycle & trade-off understanding.	Informs WORKUP's virtual company simulations and sprint backlog workflows.
Radermacher & Walia (2013, ACM SIGCSE)	Industry Deficiency Survey	Graduates struggle most with legacy codebases, unit testing, and team code reviews.	Directly validates WORKUP's focus on legacy code refactoring and PR reviews.
Kratzke (2018, IEEE Trans. Educ.)	Lightweight Sandboxes	Ephemeral container isolation removes local setup friction and prevents security leaks.	Provides architectural blueprint for path-jailed browser execution & timeout quotas.
Bogner et al. (2021, IEEE Access)	Automated Code Review SLR	Rule-based static analysis & AI reviews catch architectural drift and security bugs early.	Guides multi-rubric evaluation engine for AI Tech Lead code review feedback.
Shepherd et al. (2019, IEEE TLT)	DAG Workflow Scaffolding	Dependency-gated tasks prevent cognitive overload during complex development.	Validates prerequisite gating in WORKUP's task dependency engine (DAG).

MAJOR PROJECT REVIEW — STAGE-I

THANK YOU!

WORKUP: Virtual Professional Experience Platform

Special thanks to our Project Guide Mrs. P. Swaroopa, Head of Department Dr. M. V. Vijaya Saradhi, and the Department Review Committee for their guidance and support.

We welcome your valuable feedback and questions.